

AG-QREW on Qwen — Hackathon Reference Plan

Event: Global AI Hackathon with Qwen Cloud **Track:** Track 3 — Agent Society **Team:** n8fury (Mamdud Hasan) + foyezkabir **Deadline:** July 10, 2026, 3:00 AM GMT+6 (confirmed on Devpost). Practically: **submit by ~9:00 PM on July 9.** GMT+6 is Dhaka time, so no timezone conversion — but never plan around the literal 3 AM cutoff. **Constraint:** Claude Fable access ends July 7. All design, code generation, and documentation must be complete by end of July 7. Days after that use Opus/Sonnet for deployment, debugging, and recording only.

1. What we are building

A from-scratch rebuild of AG-QREW — a multi-agent QA automation pipeline — running on Qwen models via Alibaba Cloud Model Studio instead of Claude Code.

One sentence for the Devpost description: *An autonomous QA team of five specialized Qwen-powered agents that takes a requirements document and delivers a complete test cycle — test plan, test cases, executed Playwright scripts, API tests, filed bugs, and a sign-off report — with one human approval checkpoint.*

Why this qualifies

- Rule check:** The hackathon requires the project to *use Qwen models on Qwen Cloud at runtime*. There is no rule about what tools you use to write the code. Developing with Claude and running on Qwen is fully compliant. Roughly half of all entrants will develop with AI assistance.
- Track 3 fit:** The track asks for (a) multiple agents with distinct capabilities, (b) task decomposition and role assignment, (c) disagreement/conflict resolution, (d) a **measurable efficiency gain over a single-agent baseline**. AG-QREW's design covers a-c natively; we add (d) explicitly (see §6).
- Original repo status:** The existing repo is a set of Claude Code prompt files. It is a *reference architecture*, not portable code. The agent runtime it relies on (sub-agent spawning, tool calling, MCP integrations) belongs to Claude Code and must be rebuilt on the Qwen API. This is the core engineering work of the project — and conveniently, "we built our own multi-agent orchestration runtime" is exactly what the Technical Depth criterion (30%) rewards.

Attribution

Since foyezkabir is on the team, the original repo is our own prior art. State this plainly in the README: *"AG-QREW began as a Claude Code skill pipeline (link). For this hackathon we rebuilt it as a standalone multi-agent runtime on Qwen."* Judges reward honest evolution stories; they punish discovered ones.

2. Restructure decision: rebuild, don't port

Answer to "is restructure needed": yes — new repo, from scratch. Keep the old repo untouched as the referenced prior work.

Original component	Decision	Replacement
5 agent prompt files (.claude/commands/*.md)	Port	Become system prompts for our own agent runtime (trimmed of Claude Code-specific tool references)
Signal protocol / shared-task-list.txt	Port	Keep the file-based bus. It is human-readable, demo-friendly, and needs no broker
SFDIPOT / FEW HICcupPS heuristics	Port	Unchanged — this is domain knowledge, model-agnostic
Phase structure (0–4)	Port	Unchanged
Claude Code runtime (spawning, tool loop)	Rebuild	Our own orchestrator in TypeScript (§4)
TestRail integration	Drop	SQLite + built-in web dashboard ("Test Console")
Jira / Confluence	Drop	Bug store in SQLite + dashboard view; doc intake becomes file/paste/URL
Postman / Newman	Drop	Direct HTTP calls (fetch/axios) generated as code by qa-api-tester
Playwright via MCP	Replace	Playwright as a library, driven directly by generated specs; screenshots analyzed by Qwen-VL
GitHub Actions CI generation	Keep (cheap)	Still generate the yaml; it costs nothing and shows polish

Why drop the SaaS integrations: judges do not have TestRail/Jira/Postman accounts. A submission they cannot run scores poorly regardless of quality. Replacing them with a self-contained SQLite + dashboard stack means a judge runs `docker compose up` and sees the entire pipeline working in one command. This is a strict upgrade for judging, and the integration points can be documented as "pluggable adapters" to show production thinking.

3. Scope: what ships (MVP) vs what gets cut

MVP — must exist by end of Day 5

1. Orchestrator (QA Lead) + 4 worker agents running concurrently on Qwen
2. Signal bus (shared task list file) with live view in dashboard
3. Document intake: paste text or point at a markdown/URL requirements doc
4. Test plan generation with the single human `proceed` checkpoint
5. qa-tc-writer: structured test cases → SQLite → dashboard
6. qa-script-writer: generated Playwright specs that actually run against the demo app
7. qa-hawk: exploratory pass using Playwright + Qwen-VL screenshot analysis; bugs filed with severity + oracle
8. qa-api-tester: HTTP tests against the demo app's API from its OpenAPI spec
9. Sign-off report (PASS / CONDITIONAL / FAIL) with decision-gate numbers
10. **Demo target app** — small intentionally buggy web app shipped in the repo (\$5)
11. **Single-agent baseline mode** + comparison table (\$6)
12. Deployed on Alibaba Cloud ECS via Docker Compose

Stretch — only if Days 4-5 go smoothly

- Flakiness detection (`--repeat-each=3`)
- Accessibility (axe) soft checks
- Retest loop (Mode 3)
- Multi-language inquiry docs (shows Qwen multilingual strength)

Explicitly cut

- TestRail/Jira/Confluence/Postman adapters (mention as future work only)
- Mobile viewport testing (keep the `@mobile` tag concept in TC format, skip execution)
- Sprint health scan against Jira (no Jira)

The cut list is a feature, not a compromise. Write a "Scope decisions" section in the README explaining each one — judges read this as engineering maturity.

4. Architecture

4.1 Stack

- **Language:** TypeScript / Node.js 20 (team's primary stack; recoverable by any model or by hand)
- **Model API:** Alibaba Cloud Model Studio (DashScope), OpenAI-compatible endpoint
 - International endpoint: `https://dashscope-intl.aliyuncs.com/compatible-mode/v1`
 - Verify from Bangladesh whether the international or Singapore region console is the right one during signup — do this Day 1, not Day 4
- **Models:**
 - `qwen-max` — QA Lead (planning, gap analysis, sign-off reasoning)
 - `qwen-plus` — worker agents (cheaper, faster; test case writing, spec generation, API tests)
 - `qwen-vl-max` — qa-hawk screenshot analysis (multimodal — worth points under Technical Depth)
 - Confirm exact current model names in Model Studio on Day 1; they version frequently
- **Persistence:** SQLite (better-sqlite3) — test cases, runs, results, bugs
- **Dashboard:** Single Express server serving a small React (Vite) app — live signal feed, test case browser, bug list, sign-off report. WebSocket or SSE for live updates
- **Browser automation:** Playwright (Chromium only — cut WebKit for time)
- **Containerization:** Docker Compose — three services: `orchestrator` (agents + API + dashboard), `demo-app` (the buggy target), and a shared volume for `qa/` artifacts

4.2 The agent runtime (the real engineering)

One reusable `AgentLoop` class, instantiated five times with different configs:

```

AgentLoop {
  systemPrompt      // ported from the original .md files
  model             // qwen-max | qwen-plus
  tools[]           // function-calling schemas
  bus               // read/write signal handle
}
run(task):
  loop:
    response = qwen.chat(messages, tools)
    if tool_calls: execute each -> append tool results -> continue
    if text + done-signal: write DONE to bus, exit
    guard: max N iterations, max token budget per agent

```

Key components:

1. **Orchestrator (QA Lead):** owns Phases 0–4. Spawns workers as concurrent `AgentLoop` instances (`Promise.all` + per-agent token budgets). Polls the bus every few seconds; reacts to `SECTION-DONE`, `BLOCKED`, `DONE` signals exactly as in the original protocol.
2. **Signal bus:** append-only `qa/shared-task-list.txt` + a tiny watcher that streams new lines to the dashboard. Keep the original signal grammar verbatim — it is already well-designed and the table in the old README ports straight into the new one.
3. **Tool layer:** each tool is a typed function with a JSON schema exposed to Qwen function calling:
 - `bus_write(signal)` / `bus_read(since)`
 - `tc_store(module, cases[])` / `tc_list(module)` — SQLite
 - `bug_file(report)` — SQLite
 - `result_record(case_id, status)` — SQLite
 - `fs_write(path, content)` / `fs_read(path)` — sandboxed to `qa/`
 - `playwright_run(specPath)` — runs generated specs, returns structured results
 - `browser_snapshot(url)` — Playwright screenshot + DOM summary; screenshot goes to Qwen-VL
 - `http_request(method, url, body)` — for qa-api-tester
4. **Error handling (judged directly):** retry with backoff on Qwen API errors; per-agent iteration and token caps; a `BLOCKED` signal path that surfaces to the dashboard instead of crashing; the orchestrator survives any single worker failing and says so in the sign-off report.

4.3 The one human checkpoint

Preserve the original's defining feature: after the test plan is generated, the dashboard shows it with a **Proceed** button. Everything after is autonomous. This is your human-in-the-loop story and your demo's dramatic beat.

5. Demo target app

Ship a deliberately buggy app inside the repo (`demo-app/`): a minimal "task manager" or "shop admin" with login + one CRUD module + a small REST API with an OpenAPI spec.

Plant exactly 4 bugs, chosen so different agents catch different ones:

1. **UI bug** — error message missing on failed login (qa-hawk, FEW HICCUPPS "Claims" oracle)
2. **Functional bug** — boundary failure, e.g. 51-char title accepted when spec says max 50 (qa-script-writer negative spec)
3. **API bug** — endpoint returns 200 with an error body instead of 4xx (qa-api-tester)
4. **Data bug** — deleted item still appears in list until refresh (qa-hawk explore)

Document the planted bugs in `demo-app/PLANTED_BUGS.md` so judges can verify the pipeline found them. A pipeline that provably finds 4/4 known bugs is a far stronger demo than one pointed at a random site.

6. Single-agent baseline (Track 3 requirement)

Track 3 asks for a *measurable efficiency gain over single-agent baselines*. Most entrants will ignore this. We don't.

- Add `--mode single` : the same job runs in one monolithic agent loop with all tools
- Add `--mode society` : the normal 5-agent pipeline
- Both write a `metrics.json` : wall-clock time, total tokens, bugs found, test cases produced, specs passing
- Put the comparison table in the README and show it in the video for ~10 seconds

Expected story: the society mode parallelizes tc-writing/scripting/API testing, so wall time drops even if total tokens rise; and the single agent typically loses coherence and misses bugs on long runs. Whatever the numbers actually are — report them honestly. A judge trusts a table with one unflattering cell far more than a perfect one.

7. Repo structure

```
ag-qrew-qwen/
├── LICENSE                                # MIT — must be visible in the About section on GitHub
├── README.md                             # rewrite of the original, adapted (architecture, quickstart, metrics table)
├── HANDOFF.md                             # state + remaining tasks, for post-July-7 Claude sessions
├── docker-compose.yml
├── .env.example                           # DASHSCOPE_API_KEY, model names, ports
├── docs/
│   ├── architecture.md                   # + exported diagram PNG/SVG (submission requirement)
│   ├── signals.md                         # signal protocol table
│   └── scope-decisions.md
├── orchestrator/
│   └── src/
│       ├── agentLoop.ts
│       ├── qwen.ts                       # DashScope client wrapper, retries, budgets
│       ├── bus.ts
│       ├── tools/                         # one file per tool
│       ├── agents/                       # qaLead.ts + 4 workers (system prompts as .md, loaded at runtime)
│       ├── baseline/singleAgent.ts
│       ├── db.ts                         # SQLite schema + queries
│       └── server.ts                     # Express API + SSE + serves dashboard build
├── prompts/                              # ported & adapted agent prompts
├── dashboard/                             # Vite + React
├── demo-app/                             # buggy target app + openapi.yaml + PLANTED_BUGS.md
├── deploy/
│   ├── ecs-setup.md                     # click-by-click Alibaba Cloud instructions
│   └── deploy.sh
├── qa/                                   # runtime output, gitignored except .gitkeep
```

8. Day-by-day plan

Fable window: today (July 5) through July 7. Everything requiring design thinking lands in Days 1–3. Days 4–6 are execution with Opus/Sonnet.

Day 1 — July 5 (today): unblock everything with lag

n8fury (human tasks, ~2h):

- ☐ Register team on Devpost; confirm exact deadline + timezone
- ☐ Alibaba Cloud account + Model Studio activation; get `DASHSCOPE_API_KEY`
- ☐ Submit the hackathon credit coupon form (approval may take days — this is the single most dangerous silent blocker)
- ☐ Join Qwen Cloud Discord; skim Track 3 reference architecture in Resources
- ☐ One successful `curl` to the Qwen chat completions endpoint from your machine
- ☐ Agree split of work with foyezkabar (suggested split in §9)

With Fable (rest of day):

- ☐ Finalize this spec; lock tool schemas and DB schema
- ☐ Generate: repo skeleton, `qwen.ts` client, `agentLoop.ts`, `bus.ts`, SQLite layer
- ☐ Generate demo-app (it's small — finish it today)

Day 2 — July 6: the pipeline works end-to-end locally

- ☐ Port and adapt all five agent prompts
- ☐ Tool layer complete; QA Lead phases 0–2 running; workers spawning concurrently
- ☐ qa-tc-writer and qa-api-tester fully working (they're the simplest)
- ☐ qa-script-writer generating and executing at least one passing spec against demo-app
- ☐ Dashboard: signal feed + test case list + proceed button

Day 3 — July 7 (last Fable day): finish generation, freeze thinking

- ☐ qa-hawk with Qwen-VL screenshot analysis; bug filing end-to-end; finds the planted bugs
- ☐ Sign-off report generation
- ☐ Single-agent baseline mode + metrics.json
- ☐ Dockerfiles + docker-compose verified locally
- ☐ Generate ALL remaining text: README, architecture.md + diagram source (Mermaid → export), deploy/ecs-setup.md (written for someone with zero context), Devpost description draft, 3-minute video script with shot list, blog post draft, HANDOFF.md
- **End-of-day gate:** repo is submission-complete except cloud deployment, recordings, and bug fixes.

Day 4 — July 8 (Opus/Sonnet): deploy + proof. No slack in this day.

- ☐ Morning: ECS instance (2 vCPU / 4GB, Ubuntu 24) + security group (dashboard port, SSH); `deploy.sh` + docker compose up; full pipeline run in the cloud
- ☐ Afternoon: fix whatever the cloud broke
- ☐ Evening: record the Alibaba Cloud deployment proof (screen recording: console showing ECS + Model Studio, terminal on the instance, pipeline running). Link the proof recording AND the code file that calls the DashScope API in the README, since the rules ask for a code-file link demonstrating Alibaba Cloud usage
- ☐ **In parallel (foyezkabir):** rehearse the demo video against the local build; stage all demo data so recording on Day 5 is a single take
 - **End-of-day gate:** system runs clean on ECS and proof is recorded. If not, Day 5 morning is the hard cutoff for fixing — after noon July 9, stop fixing and record whatever works.

Day 5 — July 9: record and SUBMIT

- ☐ Morning: one full clean run on the cloud instance; demo-visible fixes only (no refactoring)
- ☐ Midday: record the 3-minute demo video; upload to YouTube (public); verify it plays logged-out
- ☐ Afternoon: make repo public; verify LICENSE shows in the About section; verify quickstart on a clean clone; final README pass
- ☐ ~6:00 PM: **submit on Devpost** — repo URL, video URL, description, track = Agent Society, deployment proof, architecture diagram. Devpost allows editing after submitting (until the deadline), so submit early and polish after
- ☐ Evening only if everything above is done: blog post for the Blog Post Prize; add its URL to the submission before 3 AM

The lost buffer day — consequences

- Day 3 (July 7) gate is now non-negotiable: repo must be genuinely code-complete, because Days 4-5 have zero room to write new features
- Blog post demoted from planned to opportunistic
- Any stretch feature not done by end of Day 3 is dead — do not carry it into Day 4

9. Division of labor (suggested)

Area	Owner
Orchestrator runtime, Qwen client, tools, baseline mode	n8fury
Agent prompt adaptation + tuning (he wrote the originals)	foyezkabir
Demo-app + planted bugs	foyezkabir
Dashboard	n8fury
Alibaba Cloud deploy + proof recording	n8fury
Video recording + editing	foyezkabir
README / docs / Devpost text	shared (generated Day 3, both review)

Adjust freely — but assign every row to exactly one name today. Unowned tasks are the ones that don't happen.

10. Submission checklist (from the rules, verbatim requirements)

- ☐ Public repo URL with all source, assets, run instructions
- ☐ Open source LICENSE file, detectable in the GitHub About section (MIT; add license via GitHub UI so it registers)
- ☐ Proof of Alibaba Cloud deployment: short recording (separate from demo video) + link to a code file in the repo showing Alibaba Cloud API usage
- ☐ Architecture diagram (visual; export the Mermaid to PNG/SVG — don't rely on GitHub rendering alone)
- ☐ ~3 minute demo video, public on YouTube/Vimeo/Facebook
- ☐ Text description of features and functionality
- ☐ Track identified: **Track 3 — Agent Society**
- ☐ Optional: blog/social post URL for the Blog Post Prize

11. Video plan (3 minutes)

1. **0:00-0:20** — Problem: a sprint QA cycle is days of manual work across test plans, cases, scripts, bug reports
2. **0:20-0:50** — Paste a requirements doc, agent generates the test plan, click **Proceed** (the one human decision)

3. **0:50-1:50** — The society at work: dashboard split-screen — live signal feed scrolling, test cases filling in, Playwright running headed for 5 seconds, a bug getting filed with a Qwen-VL-analyzed screenshot
4. **1:50-2:20** — Results: sign-off report; 4/4 planted bugs found; open PLANTED_BUGS.md to prove it
5. **2:20-2:45** — Society vs single-agent metrics table
6. **2:45-3:00** — Architecture diagram, one sentence on Qwen models used, close

Rules of the recording: no dead time, no typing on camera (pre-stage everything), voiceover written in advance, 1080p.

12. Risks and cut lines

Risk	Mitigation
Model Studio signup/credits blocked from BD	Do Day 1. Fallback: pay-as-you-go is cheap for this scale; credits are a bonus, not a dependency
Qwen function calling behaves differently than expected	Day 1 curl test includes a tool-call round trip, not just a hello-world
Multi-agent chaos under deadline	Agents are just concurrent loops over one tested class; if concurrency misbehaves, run workers sequentially — the signal bus makes this a one-line change and the demo barely changes
qa-hawk (hardest agent) slips	Cut line: hawk does smoke mode + planted-bug hunting only, no full SFDIPOT explore
Playwright in Docker on ECS	Use the official mcr.microsoft.com/playwright base image; test locally in Docker on Day 3, not on ECS on Day 4
Deadline: July 10, 3:00 AM GMT+6	Confirmed. Submit by ~6 PM July 9; Devpost permits post-submission edits until the deadline, so early submission costs nothing

Priority order if time collapses: working pipeline on cloud > video > README > baseline metrics > dashboard polish > stretch features. A deployed system with a clear video beats a feature-complete repo that never left localhost.

13. After July 7 — working with Opus/Sonnet

- HANDOFF.md** is the contract: current state, how to run everything, exact remaining tasks, known issues. Fable writes it Day 3; keep it updated.
- Give the next model narrow tasks ("this deploy step failed with this error"), not open-ended ones ("finish the project"). All open-ended thinking should already be done.
- Keep this document and HANDOFF.md in the repo root so any session can be bootstrapped with two file reads.